MemDB2 C DLL Benchmark: Database Exceeding RAM

mdb2bench, runs of 12–13 November 2026 · x64 Release DLL · 8.44 million records · 4 readers + 4 writers

This is an additional run, with 10–15% of the database outside RAM, in the page file on disk. Its purpose is to characterise MemDB2's behaviour when it finally exhausts memory, not its normal performance. For that, see MemDB2_Benchmark_Results_2026-11-12.pdf, whose 1M-row runs fit comfortably in RAM and are used for comparison throughout.

The machine was only marginally usable throughout these runs. General responsiveness was poor enough that a human user would immediately notice that some remedial action was needed, such as freeing memory, adding RAM or moving data elsewhere. The figures below describe a system in that state.

÷34–59
mixed-workload throughput against the same runs in RAM (564–847 txn/s)

×12–49
mean latency of isolated operations that touch random rows

11–17 s
worst single transaction in each mixed run

×1
find earliest expiry: unchanged, because it only touches pages that stay in RAM

Summary

- **MemDB2 kept working correctly.** Every run completed with no errors and no transactions abandoned. Concurrency clashes appeared for the first time in any run (in the rws run's expiry sweeps), because transactions now take long enough to overlap, and every one succeeded on retry.
- **Throughput fell 34–59 times.** The mixed workload ran 564–847 transactions/s, against 24,000–33,000 in RAM. Writers managed 67–227 transactions/s.
- **Operations on random rows became 12–49 times slower;** a find by ID takes about 2 ms instead of 38 µs, an insert 4–9 ms instead of 0.2 ms. Tails reached seconds: p99.99 up to 3.7 s for isolated operations, and individual transactions up to 16.5 s in the mixed phase.
- **Locality decides the cost, not size.** Find earliest expiry, which walks only the leftmost path of the Expiry index, ran at 28 µs, exactly as in RAM. Rows and index nodes reached through random RowIDs and Ids are scattered across the whole heap, so each access risks a page fault.
- **Maintenance slowed most.** The synchronous checkpoint after loading took 835 s, about 40 times its in-RAM rate. Reopening took 13–52 minutes instead of an expected 4–5. Loading ran at 11,400 rows/s until memory ran out, then at 310 rows/s.
- **CPU per transaction rose 3–7 times,** mostly time spent handling page faults, which Windows charges to the faulting thread.

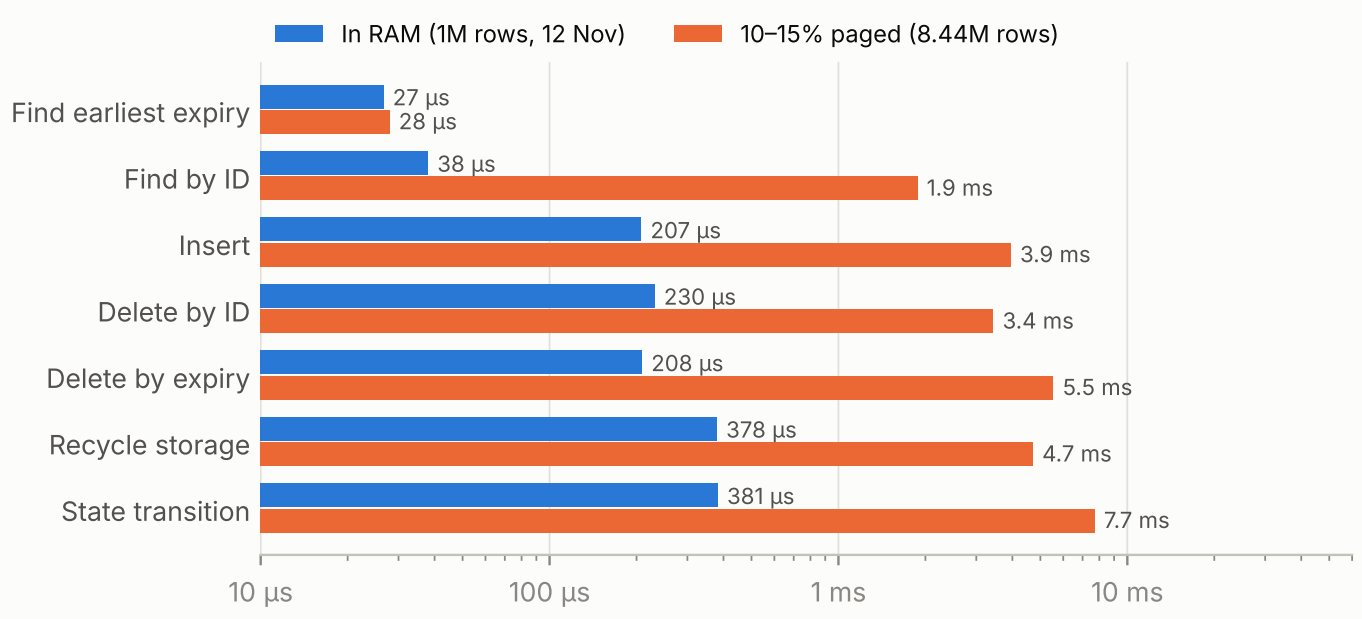
Test setup

Machine	Laptop, Intel Core i7-3632QM (Ivy Bridge, 2012), 2.2 GHz, 4 cores / 8 threads, 16 GB RAM, SSD; invariant TSC (2.195 GHz)
Database	MemDB2 C DLL, x64 Release, assertions off; journal flow control at 128 MB of queued changesets. All calls go through the C API shim
Data	8,440,000 rows of about 128 bytes (Id, Expiry, State, Payload blob); unique index on Id, index on Expiry. 12.8 GiB (13,130 MiB) of process memory after loading (1,631 bytes per row), of which an estimated 10–15% was in the page file during the runs
Why 8.44M rows	A 21M-row load was stopped at this point, when paging had slowed it to about 140 rows/s. The row count was set to match, so the database sits just past the point where it no longer fits
Procedure	`big_test.bat`: loaded once with 8 threads and checkpointed; before each run the database folder was copied and reopened, so all three runs start from the same data
Isolated phase	Each of 7 operations alone on one thread, 200,000 transactions each
Mixed phase	4 reader + 4 writer threads, unpaced, 1,800 s after a 60 s warm-up; same operation mix as the in-RAM runs
Runs	read (READ_SHARED readers), rws (READ_WRITE_SHARED readers), rws-ckpt (as rws, with back-to-back asynchronous checkpoints)

Isolated operations

Figure 1. Mean latency per transaction, isolated, rws run

Logarithmic scale. In RAM (1M rows) against 10–15% paged (8.44M rows).



Isolated, one thread	mean, in RAM	mean, paged	slower by	mean, 3 paged runs	p99.99, paged	max, paged
Insert	207 µs	3.94 ms	×19	3.94 ms–8.89 ms	1.74 s	7.38 s
Find by ID	38 µs	1.87 ms	×49	1.87 ms–2.54 ms	178 ms	6.30 s
Find earliest expiry	27 µs	28 µs	×1	28 µs–32 µs	515 µs	2.13 ms
Delete by ID	230 µs	3.42 ms	×15	3.12 ms–6.83 ms	1.10 s	7.92 s
Delete by expiry	208 µs	5.52 ms	×27	5.52 ms–13 ms	2.81 s	5.49 s
State transition	381 µs	7.68 ms	×20	6.46 ms–15 ms	3.03 s	7.70 s
Recycle storage	378 µs	4.69 ms	×12	4.69 ms–9.06 ms	2.51 s	5.15 s

Every operation that reaches a row by its Id, or by its RowID through the internal index, is 12–49 times slower. A find by ID descends the Id index to a random leaf and then reads a random row, and either can be on disk. Writes touch more pages (the row, its versions, and the Id, Expiry and internal indexes), so they fault more in absolute terms, but their in-RAM cost was already higher, so the ratio is smaller: ×12–27, against ×49 for a find. The three paged runs vary by up to a factor of 2.3 for the same operation, against 5% in RAM, because how much of the heap is resident depends on what else the machine was doing.

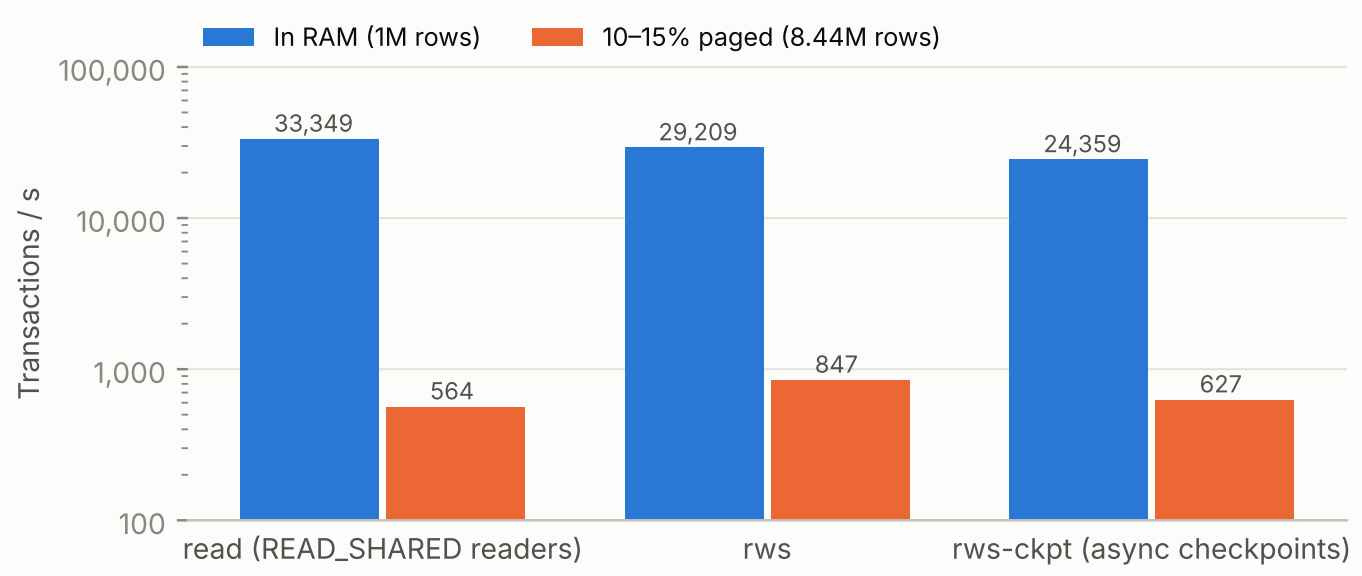
Find earliest expiry is the exception. It always takes the lowest entry in the Expiry index, so it touches the same few nodes every time, and they stay in RAM. Its 28 µs matches the in-RAM figure.

For operations on random rows, tails are measured in seconds: p99.99 is 0.2–3.7 s and the worst cases 3–11 s. CPU cycles per write rose from about 450,000–840,000 to 1.3–4.0 million, the extra being the faulting thread's time in the kernel.

Mixed workload

Figure 2. Mixed-workload throughput

Logarithmic scale. Transactions per second over the 30-minute measurement.

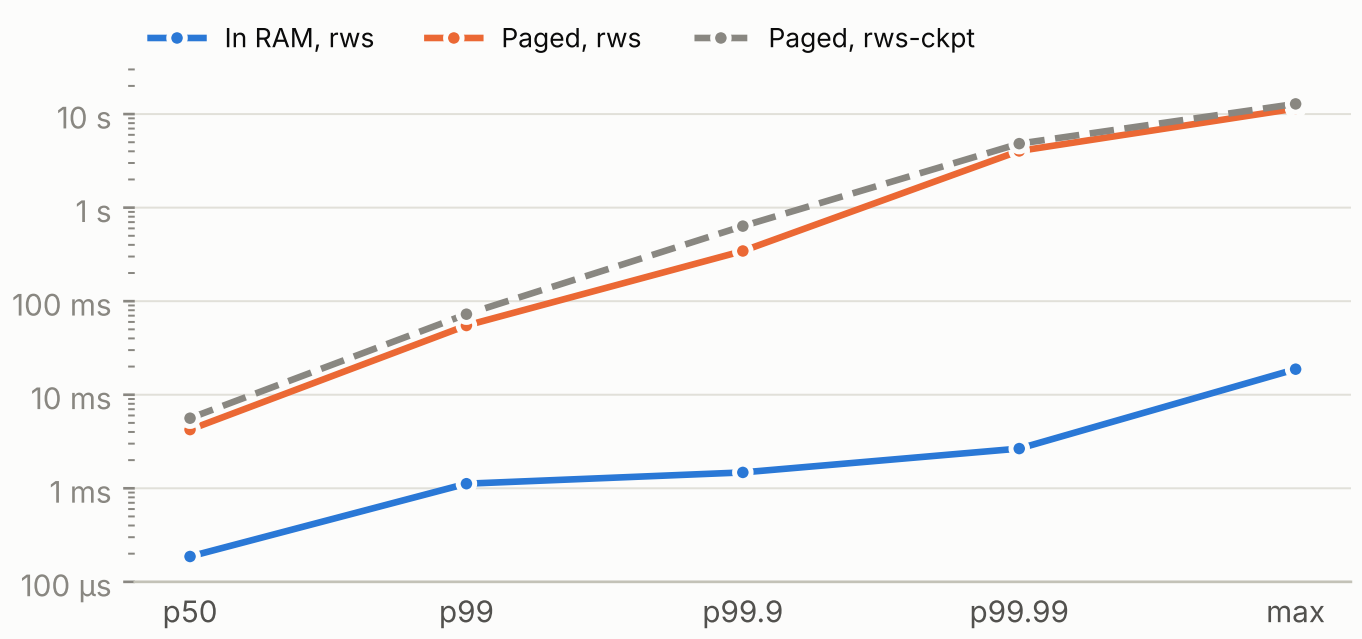


Mixed, 4 + 4 threads	txn/s, in RAM	txn/s, paged	change	writers/s	readers/s	p50	p99	p99.99	max
read	33,349	564	÷59	67	497	2.90 ms	122 ms	5.32 s	16.54 s
rws	29,209	847	÷34	227	620	4.24 ms	55 ms	4.04 s	11.47 s
rws-ckpt	24,359	627	÷39	176	452	5.61 ms	73 ms	4.83 s	12.85 s

Throughput fell 34–59 times. Medians rose from about 0.2 ms to 3–6 ms, and every run had transactions taking 11–17 s. Unlike in RAM, the read run was slower than the rws run here; it was the first run after the copy and its database took longest to reopen, so it probably started with less of its data resident. With one run each, that difference shouldn't be read as an effect of the access modes.

Figure 3. Latency percentiles, all mixed-phase transactions

Logarithmic scale. rws in RAM; rws and rws-ckpt paged.



Concurrency clashes. The rws run had 10,769 clashes on delete by expiry and 2,123 on recycle, the first in any run. With transactions taking milliseconds to seconds, sweeps that start close together often go for the same earliest rows. All were retried successfully.

Checkpoints. In the rws-ckpt run only one asynchronous checkpoint started during the 31-minute phase, and it finished after the measurement ended, so every transaction overlapped it. Throughput was 26% lower than the rws run, similar to the 17% cost in RAM.

Fewer misses. Only 7% of finds by ID targeted deleted rows, against 43% in RAM, because far fewer rows were inserted and deleted during the phase.

Loading, checkpoints and reopening

Operation	Paged, 8.44M rows	In RAM, 1M rows	In RAM, scaled to 8.44M	Slower by
Load, first 90% (7.6M rows)	664 s (11,400 rows/s)	61–62 s per 1M (16,200–16,400 rows/s)		×1.4
Load, last 10% (0.84M rows)	2,732 s (310 rows/s)		as above	×53
Load, all 8.44M rows	3,396 s (2,485 rows/s)		as above	×6.6
Synchronous checkpoint after load	835 s	2.5 s	21 s	×40
Reopen (read checkpoint, build indexes)	765–3,116 s	31 s	263 s	×3–12
Count rows in storage order (C API)	51–82 s	2.3 s	20 s	×2.6–4.2

The load ran at 11,400 rows/s, about 70% of the 1M in-RAM rate, until about 7.6M rows; the last 0.84M rows then took 46 minutes. The checkpoint walks every row and was 40 times slower than in RAM. Reopening read the checkpoint back and rebuilt the indexes in about 13–14 minutes for two runs and 52 minutes for the first, against about 4–5 minutes if it scaled from 1M rows in RAM. Counting the rows through the C API took 51–82 s.

Memory per row was 1,631 bytes, close to the 1,625 of the previous single-threaded load. At 1M rows the parallel load had used 1,906 bytes per row, so its extra memory appears to be a fixed overhead of a few hundred MB rather than a cost per row.

Interpretation

- **The failure is gradual and correct, but steep.** With only 10–15% of the database out of RAM, throughput dropped by more than an order of magnitude and worst-case latency reached seconds. In practice a deployment must keep the whole working set in RAM; there is no useful "slightly over" region.
- **Random RowIDs make it worse than it needs to be.** RowIDs are random GUIDs, so consecutive inserts land in unrelated parts of the internal index, and a lookup by RowID can touch any part of it. Sequential or partly sequential RowIDs would keep recently inserted rows and their index nodes together, and make paging much less random.
- **Maintenance operations are the most exposed.** Checkpointing and reopening touch every row, so they pay for every page that is out, and their times grew far more than the per-transaction costs.

Caveats

- **Old hardware, and a stressed machine.** The laptop dates from 2012, and during these runs it was only marginally usable. Paging behaviour depends heavily on the disk, on Windows' memory manager and on other programs, so these figures characterise this machine in this state rather than MemDB2 in general.
- **The share on disk is an estimate.** 10–15% is inferred from process memory against RAM in use by Windows and other programs; it wasn't measured during the runs, and it varied over time.
- **Size also differs from the in-RAM runs.** The comparison is between 8.44M and 1M rows, so trees are about 3 levels deeper. That accounts for perhaps 15% more work per lookup in RAM, small next to the paging effect.
- **Expiry processing wasn't exercised in the mixed phase,** as in the in-RAM runs: records expire 11.7M inserts after their own, and at most 167,000 ids were issued per phase.
- **One run per configuration,** and the three paged runs differ more from each other than the in-RAM runs did.

Appendix: data

Isolated phase, read run

Operation	txn/s	mean	p50	p99	p99.9	p99.99	max	thread cycles
Insert	112	8.89 ms	5.73 ms	39 ms	159 ms	3.12 s	9.87 s	2,372k
Find by ID	394	2.54 ms	1.67 ms	9.79 ms	69 ms	1.33 s	3.47 s	579k
Find earliest expiry	31,300	32 µs	28 µs	85 µs	200 µs	784 µs	7.11 ms	69k
Delete by ID	146	6.83 ms	5.31 ms	24 ms	126 ms	2.81 s	7.24 s	2,018k
Delete by expiry	79	13 ms	9.67 ms	41 ms	271 ms	3.52 s	9.95 s	3,140k
State transition	67	15 ms	11 ms	46 ms	348 ms	3.70 s	11.20 s	4,000k
Recycle storage	110	9.06 ms	6.87 ms	31 ms	166 ms	3.18 s	7.27 s	2,951k

Isolated phase, rws run

Operation	txn/s	mean	p50	p99	p99.9	p99.99	max	thread cycles
Insert	254	3.94 ms	1.37 ms	27 ms	75 ms	1.74 s	7.38 s	1,931k
Find by ID	535	1.87 ms	1.00 ms	13 ms	35 ms	178 ms	6.30 s	525k
Find earliest expiry	35,800	28 µs	27 µs	51 µs	112 µs	515 µs	2.13 ms	61k
Delete by ID	292	3.42 ms	2.39 ms	14 ms	57 ms	1.10 s	7.92 s	1,430k
Delete by expiry	181	5.52 ms	4.06 ms	21 ms	95 ms	2.81 s	5.49 s	1,981k
State transition	130	7.68 ms	5.61 ms	26 ms	149 ms	3.03 s	7.70 s	2,907k
Recycle storage	213	4.69 ms	3.52 ms	17 ms	62 ms	2.51 s	5.15 s	2,191k

Isolated phase, rws-ckpt run

Operation	txn/s	mean	p50	p99	p99.9	p99.99	max	thread cycles
Insert	164	6.08 ms	4.30 ms	23 ms	92 ms	2.69 s	7.39 s	2,072k
Find by ID	522	1.92 ms	1.15 ms	8.36 ms	26 ms	375 ms	7.20 s	534k
Find earliest expiry	35,000	28 µs	27 µs	56 µs	112 µs	508 µs	2.49 ms	62k
Delete by ID	321	3.12 ms	2.21 ms	11 ms	47 ms	879 ms	8.08 s	1,341k
Delete by expiry	160	6.26 ms	4.66 ms	21 ms	85 ms	2.84 s	6.90 s	2,060k
State transition	155	6.46 ms	4.54 ms	22 ms	112 ms	2.94 s	11.08 s	2,502k
Recycle storage	191	5.24 ms	4.12 ms	16 ms	77 ms	2.54 s	7.51 s	2,327k

Mixed phase, read run (readers READ_SHARED)

Operation	txns	mean	p50	p99	p99.9	p99.99	max	misses	clashes
Insert	44,489	42 ms	22 ms	208 ms	3.33 s	5.01 s	13.76 s	0	0
Find by ID	841,902	8.42 ms	2.81 ms	81 ms	294 ms	4.10 s	11.66 s	58,412	0
Find earliest expiry	93,545	4.62 ms	56 µs	76 ms	235 ms	3.30 s	8.25 s	0	0
Delete by ID	22,331	91 ms	68 ms	409 ms	5.01 s	9.78 s	13.87 s	1,568	0
Delete by expiry	43,026	53 ms	45 ms	208 ms	3.49 s	12.35 s	16.54 s	43,026	0
State transition	11,142	101 ms	75 ms	417 ms	5.63 s	13.82 s	14.33 s	778	0
Recycle storage	4,447	51 ms	48 ms	203 ms	2.08 s	3.63 s	3.63 s	4,447	0

Mixed phase, rws run (readers READ_WRITE_SHARED)

Operation	txns	mean	p50	p99	p99.9	p99.99	max	misses	clashes
Insert	149,214	20 ms	13 ms	74 ms	1.27 s	7.22 s	10.98 s	0	0
Find by ID	1,046,574	7.03 ms	3.82 ms	33 ms	214 ms	3.21 s	8.49 s	75,723	0
Find earliest expiry	116,287	1.49 ms	129 µs	13 ms	41 ms	512 ms	4.56 s	0	0
Delete by ID	74,728	32 ms	23 ms	115 ms	1.48 s	5.69 s	10.72 s	5,249	0
Delete by expiry	149,774	4.55 ms	211 µs	41 ms	138 ms	2.94 s	8.46 s	140,979	10,769
State transition	37,315	37 ms	26 ms	136 ms	2.90 s	5.93 s	11.47 s	2,640	0
Recycle storage	15,044	5.17 ms	226 µs	66 ms	204 ms	803 ms	2.78 s	14,715	2,123

Mixed phase, rws-ckpt run (one async checkpoint running throughout)

Operation	txns	mean	p50	p99	p99.9	p99.99	max	misses	clashes
Insert	118,131	27 ms	17 ms	111 ms	2.66 s	7.40 s	11.77 s	0	0
Find by ID	762,952	9.65 ms	5.14 ms	47 ms	386 ms	3.91 s	11.17 s	54,713	0
Find earliest expiry	84,773	1.74 ms	107 µs	18 ms	67 ms	1.02 s	3.28 s	0	0
Delete by ID	59,161	41 ms	29 ms	162 ms	3.00 s	8.19 s	12.85 s	4,255	0
Delete by expiry	110,557	3.75 ms	134 µs	24 ms	119 ms	2.97 s	6.80 s	110,557	0
State transition	29,515	48 ms	32 ms	182 ms	3.24 s	11.37 s	11.58 s	2,118	0
Recycle storage	11,883	4.02 ms	153 µs	27 ms	130 ms	2.94 s	3.06 s	11,883	0

Source: `Common\MemDB2\Bench\results_big.csv` and `results_big_execution.log`, 12–13 November 2026. In-RAM figures: MemDB2_Benchmark_Results_2026-11-12.pdf.